ORM solution : JPA & Hibernate

Fabrice Jouanot
Nicolas Marinho

Plan du cours

- Approche ORM
- Mapping basique
- Mapping relations
- Mapping héritage
- Le retour du DAO: Repositories via Spring DATA
- avec son exemple fil rouge…

Introduction

Concept des ORM ?

Problématique

- Java
  - Java est reconnu comme plateforme multi-tier (J2EE)
  - Langage à objets
- Lien Java – SQL
  - La BDR = source de persistance
  - Java = application
  - JDBC = Beaucoup de travail fastidieux de bas niveau sans relation direct avec les problèmes métiers !
- Mapping Objet-Relationnel = abstraction d'une BDR pour simuler une BDOO
- Approche DAO = encore beaucoup de code & SQL dans les DAO
- ORM = manipulation des données avec des objets + gestion de la persistance dans une BDR, mais sans coder !

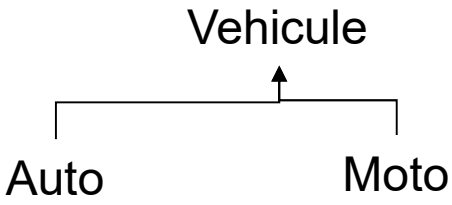
Cohabitation objet - relationnel

- La persistance des objets restent un problème
  - La sérialisation n'est pas une solution: accès tout ou rien (sauvegarde de l'état du graphe d'objets)
  - Le modèle relationnel ne peut pas capturer directement les paradigmes du modèle à objets
- Le modèle relationnel reste LA solution de persistance, mais comment résoudre
  - Le problème de granularité
  - Le problème d'héritage (sous type)
  - Le problème d'identification
  - Le problème d'associations
  - Le problème de navigation dans un graphe d'objets

Le problème de granularité

- lié à la limite de taille des objets manipulés: exemple de l'adresse (pour un utilisateur)
  - Une classe Adresse en Java qui peut:
    - être décomposée en des éléments à granularité plus fine (numéro, rue, ville, etc.)
    - être référencée dans différents objets
  - Une table Adresse n'est pas possible (règles de normalisation)
    - utiliser un type utilisateur Adresse (extension SQL)
    - utiliser des attributs associés à l'utilisateur (peu flexible)
- Le problème est bien connu et les choix Adhoc.

Le problème d'héritage



- Java permet de définit aisément cette relation de sous-typage
  - un simple lien d'héritage
  - qui autorise le polymorphisme
- Une BD SQL ne permet pas directement ce type de modélisation (ou en Relationnel Objet)
- Les choix d'implémentation sont encore bien connus mais la aussi Adhoc.

Le problème d'identification

- Le modèle à objet est plus riche en matière d'identité
  - identité d'objets (même emplacement mémoire)
  - égalité d'objets (même objet de surface, opérateur equals())
- Une BD SQL se limite à la définition d'une clé primaire (ou identifiant)
  - Comparer des objets devient délicat
  - L'existence d'objets persistants et non persistants complexifie le problème

Le problème d'associations

- En java une association est une référence vers un ensemble d'objets
  - Personnes et Téléphones / Adresses sont liés
  - Relation explicitement bidirectionnel
- En BD SQL l'association est modélisé par les clés étrangères (sur des clés primaires)
  - Personnes et Téléphones sont stockés dans leur propre table
  - la relation n'est pas bidirectionnel: many-to-one en général
  - Une relation bidirectionnelle implique
    - soit la création d'une table de liaison : artificielle et sans intérêt au niveau métier (sauf lien n-n) !
    - Soit le reconstruction via requêtes.

Le problème de navigation

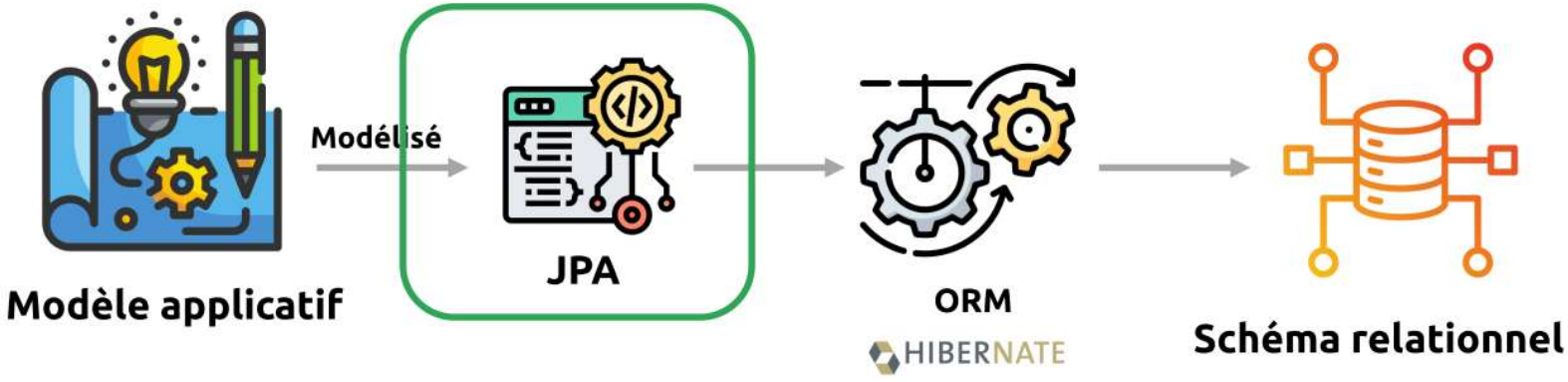
- En java, il est naturel de naviguer dans le graphe d'objets pour construire les objets résultats
  - notation chainée (tel.getIdPersonne().getNom())
  - pas de surcoût à la navigation
- En relationnel, la notion de navigation est remplacée par la notion de jointure

```
select *
from telephone tel
left outer join personne p on tel.Id_presonne=p.id
where tel.id=680010203
```

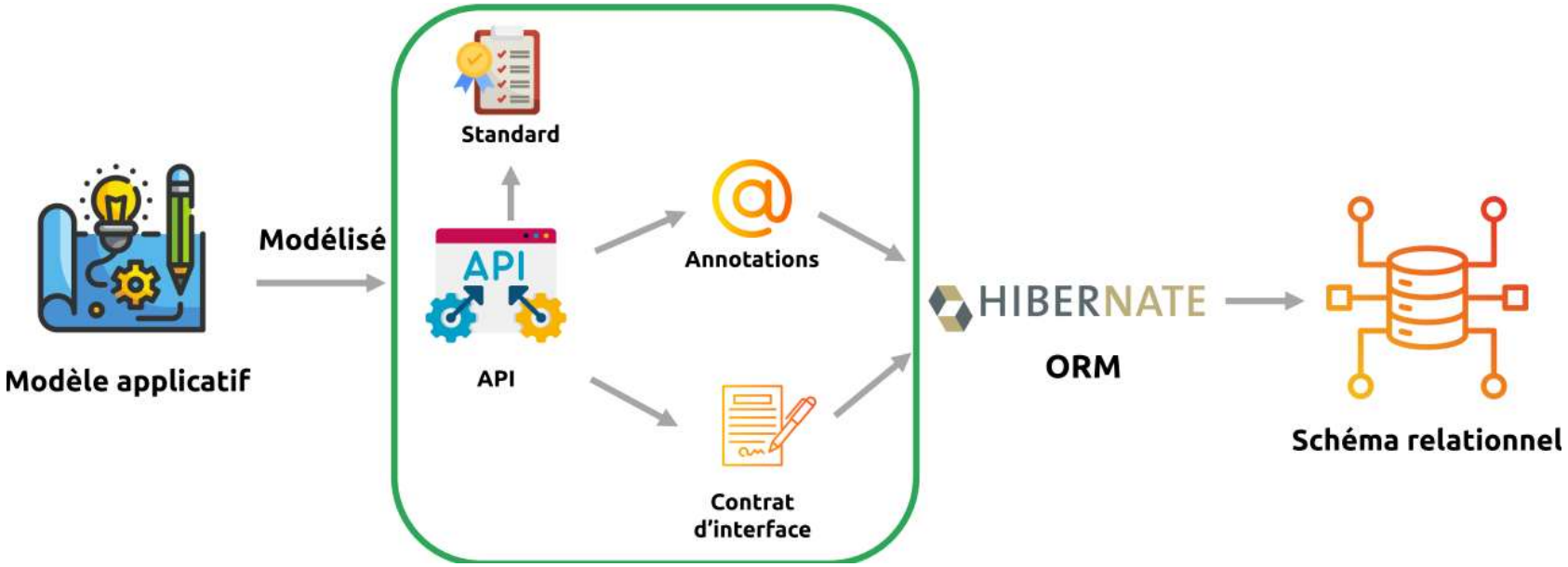
  - le passage objet > BD SQL peut générer une requête de jointure pour chaque nœud du graphe (*n+1 selects problem*)

Norme JPA

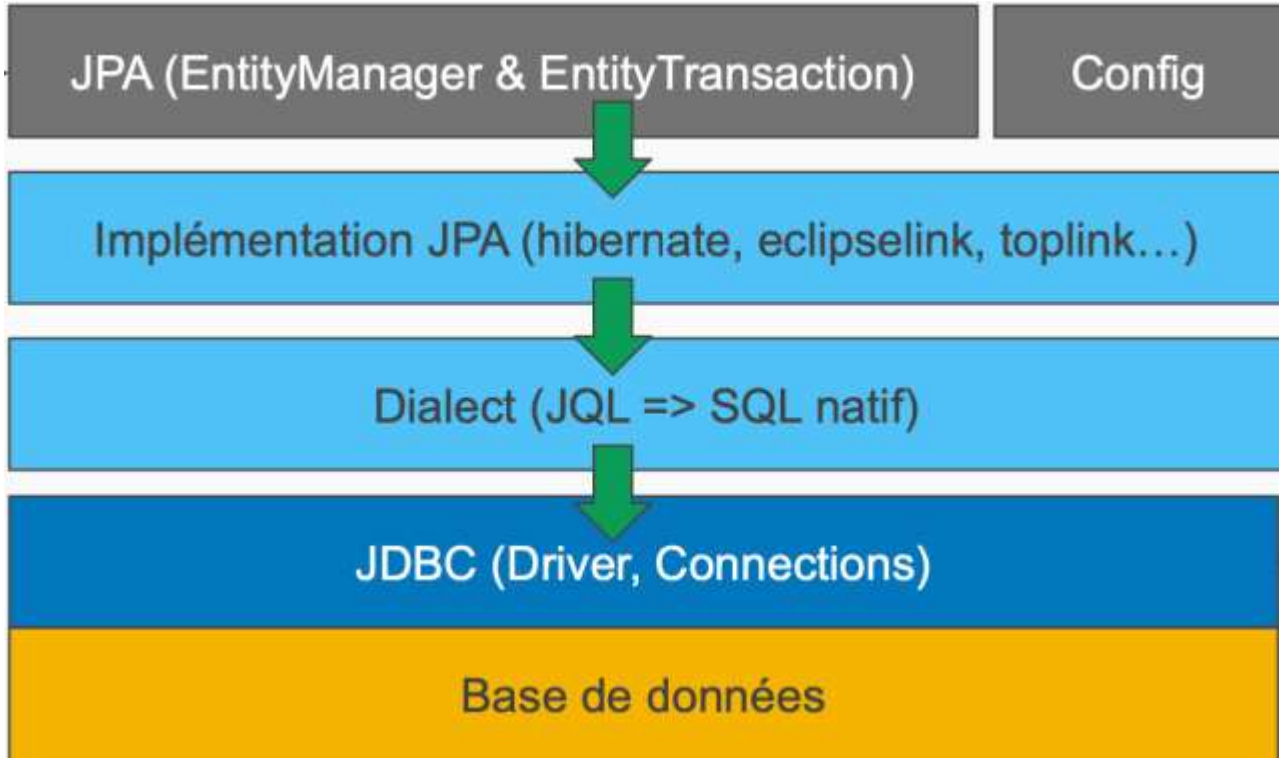
- Plusieurs solutions ORM sont apparues,
  - Chacun possède son standard
  - Hibernate = acteur principal
- JPA impose un standard JAVA
  - Pour le mapping, l'accès aux données, l'interrogation
  - Nécessite une implémentation: Hibernate le plus souvent



Java Persistante API

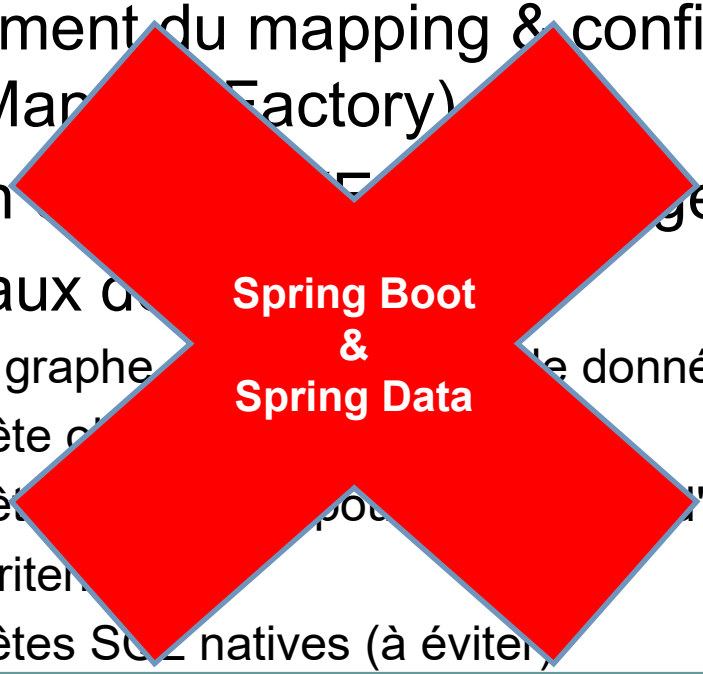


JPA: Techniquement



JPA: Logiquement

- Schéma conceptuel (UML)
- Construction du modèle de données (POJO)
- Définition du mapping (annotation ou XML)
- Chargement du mapping & configuration (EntityMan Factory)
- Session ger)
- Accès aux d
  - Via le graphe le données (getter & setter)
  - Requête c
  - Requêt l'abstraction
  - API Criter
  - Requêtes SQL natives (à éviter)



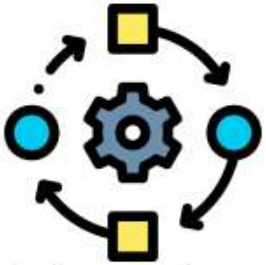
Mapping basiques

Automatisation ou Adaptation



JPA

2 Paradigmes

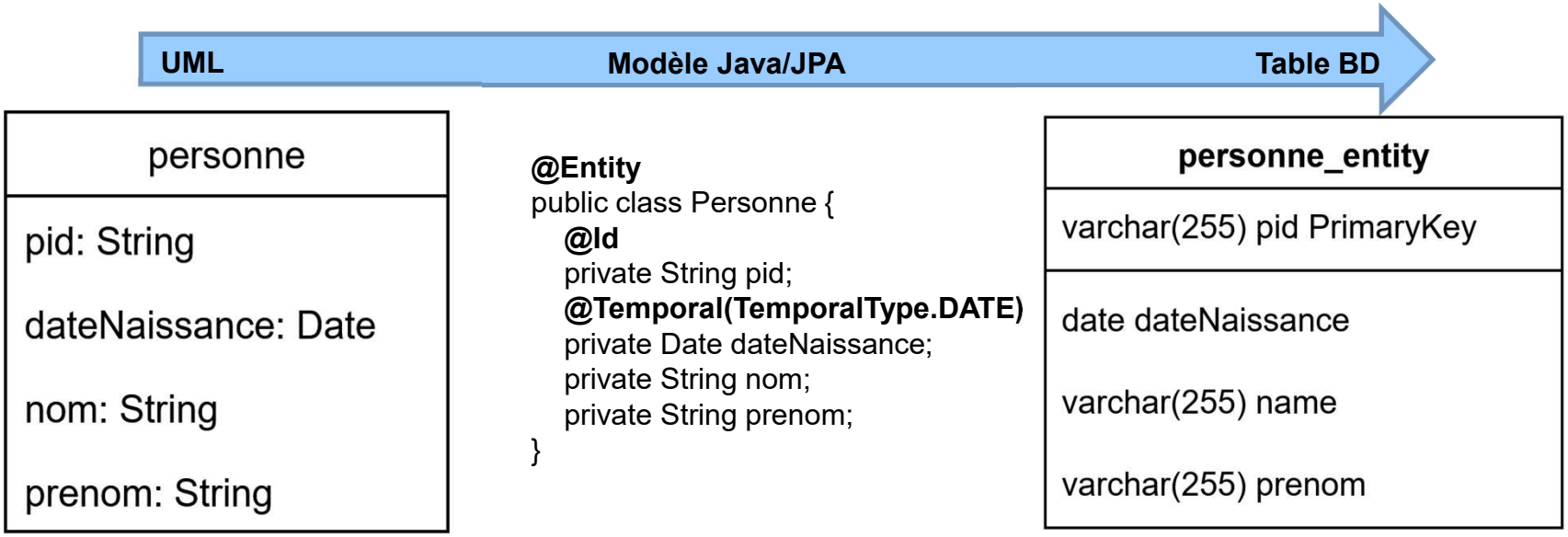


S'adapte à un schéma existant
(*validate*)



Crée un schéma en fonction du mapping
(*create*)

Définition d'une entité sérialisable



- On est ici en mode « Create »
- @Entity sérialise une classe
- @Id précise l'inditifiant / clé primaire
- Par défaut les attributs sont sérialisés

Définition d'une entité sérialisable



personne
pid: String dateNaissance: Date nom: String prenom: String

```
@Entity
@Table(name="personne")
public class Personne {
   @Id
   private String pid;
   @Temporal(TemporalType.DATE)
   @column(name="pdNaiss",
                nullable=false)
   private Date dateNaissance;
   @column(name="pnom",
                nullable=false)
   private String nom;
   @column(name="pprenom")
   private String prenom;
}
```

personne
varchar(255) pid PrimaryKey
date pNaiss NOT NULL varchar(255) pname NOT NULL varchar(255) pprenom

- On ici en mode « Validate »
- @column précise les paramètres des attributs de la table

Classe intégrable (embedded)

- Représentation d'une relation 1-1
  - En imposant la persistance sous forme d'attributs au sein d'une table d'entité
  - Annotation @Embeddable & @Embedded
- Exemple entre une personne et les détails d'une personne (lien fort)



```
@Embeddable
public class PersonDetail {
  @Temporal(TemporalType.DATE)
  private Date dateNaissance;
}
```

```
@Entity
public class Personne {
  @Id
  private Integer pid;
  private String nom;
  private String prenom;

  @Embedded
  private PersonneDetail personDetail;
}
```

Table BD: Personne-entity(pid, nom, prenom, dateNaissance)

Enumération

- JPA gère les énumérations: @enumerated
  - Soit sous forme ordinal (valeur des éléments ramenés à un ensemble de 0 à n)
  - Soit sous forme originelle (String)

```
public enum Genre { FEMME, HOMME }

@Entity
public class Personne {
  …
  @Enumerated{EnumType.ORDINAL }
/* ou EnumType.STRING */
  private Genre genre;
  …
}
```

Les champs simples persistables

- Les attributs static, final ou transient ne sont pas *persistables* !
- Les types simples sont tous persistables: Integer, Long, Short, Float, Double, Character, String, Byte, Boolean, Date, Time, TimeStamp, etc.
- Annotation @basic pour expliciter la perstistence et @column pour customiser le mapping avec les paramètres principaux:
  - name: changer le nom de colonne dans la table
  - unique/nullable: configurer le type SQL
  - insertable/updatable: par défaut à true
  - columndefinition: renseigner le type de la colonne

Les champs simples persistables

- Pour les types dates une annotation spécifique: @temporal pour spécifier:
  - Type calendaire: TemporalType.DATE (stocke le jour)
  - Type temporel: TemporalType.TIME (stocke l'heure)
  - Type système: TemporalType.TIMESTAMP (stocke jour&heure)
- @basic est optionnel
  - Autre manière de préciser le caractère non nullable
  - Et le chargement à la demande d'un attribut (peu utile sur type Simple)

```
@entity
public class Personne {
    @basic(optional=false)
    private Integer id;
    private String nom;
    private String prenom;

    @column(name="date_naissance")
    @Temporal(TemporalType.DATE)
    private Date dateNaissance;
    …
}
```

Clé primaire

- Une annotation doit spécifier l'attribut qui joue le rôle de clé primaire: @Id
  - On évitera les clés composites (non naturel à définir)
  - Deux paramètres importants pour le modèle de génération auto de clé: *generationType* & *generator*
- GenerationType.AUTO
  - Gérée par l'ORM
  - Avec une implem Hibernate,

table de séquence unique

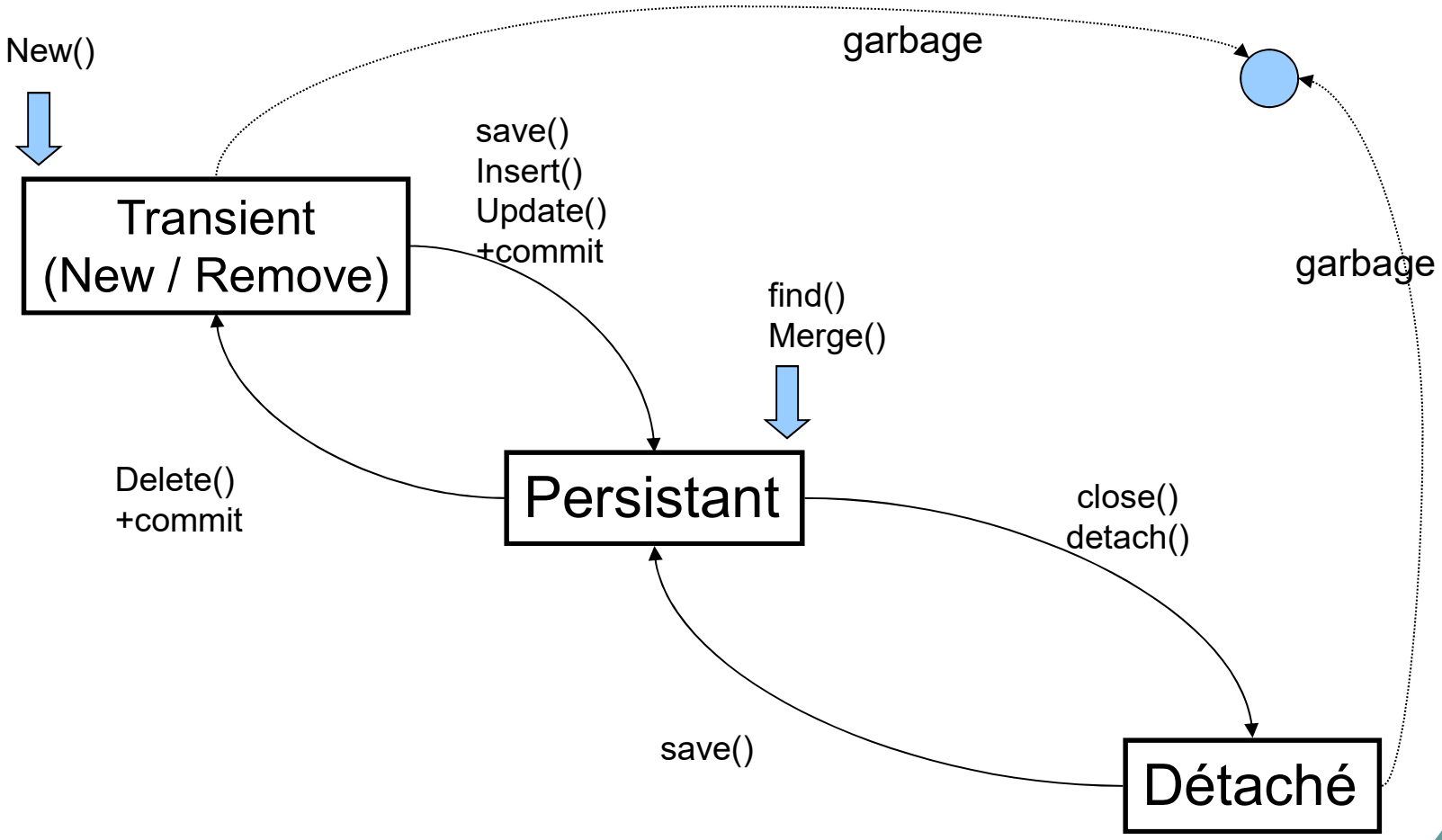
pour tous les Ids

```
@Entity
public class Personne {
  @Id
  @GeneratedValue(strategy=
                GenerationType.AUTO)
  private Integer id;
  …
}
```

Clé primaire

- GenerationType.IDENTITY
  - Gérée par le SGBD, peuplement incrémental spécifique au SGBD
  - Chaque table possède son propre incrément
  - Pas compatible avec tous les SGBD (Oracle 11g par exemple)
- GenerationType.TABLE / SEQUENCE
  - Extra code pour générer des séquences spécifiques à chaque table
  - A utiliser si nécessaire

Cycle de vie / états des objets



!Chercher un objet détaché crée un clone!

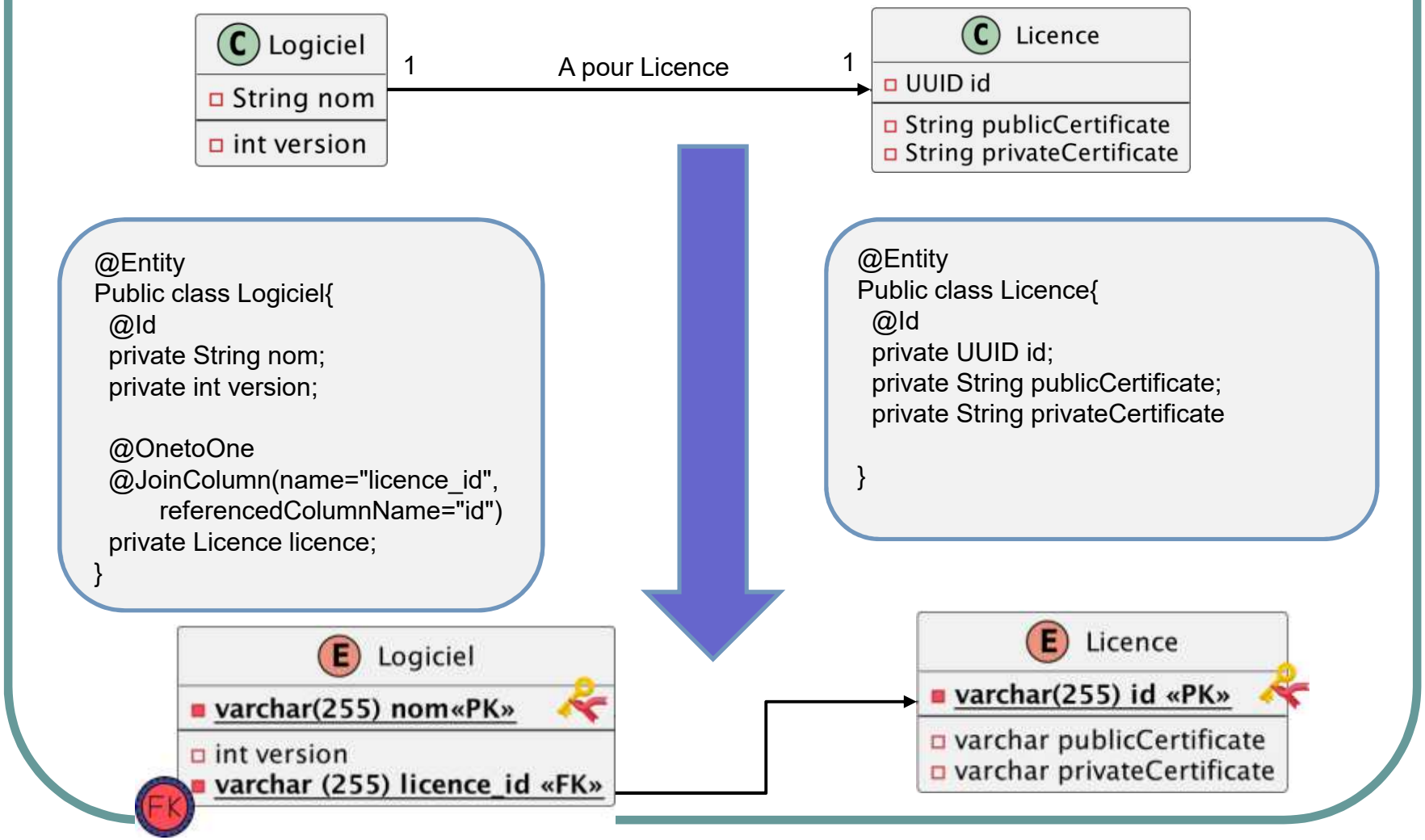
Mapping: Les relations / associations

4 types de relation

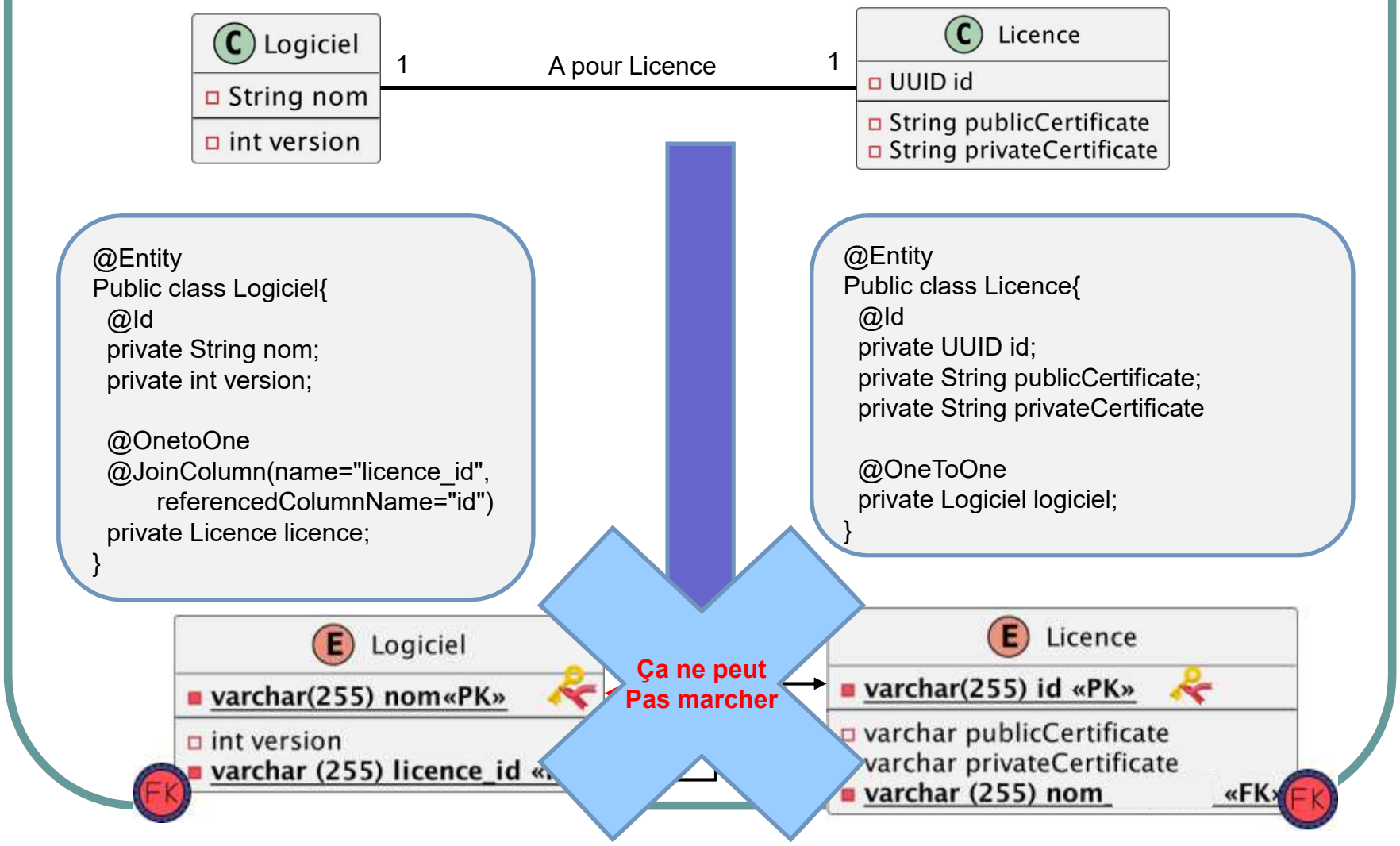
- @OneToOne (1:1)
- @ManyToOne (N:1)
- @OneToMany (1:N)
- @ManyToMany (N:N)

- Les relations peuvent être:
  - Unidirectionnelle
  - Bidirectionnelle

@OneToOne, unidirectionnelle



@OneToOne, bidirectionnelle



Navigation dans le graphe d'objets: **Notion de relation inverse**

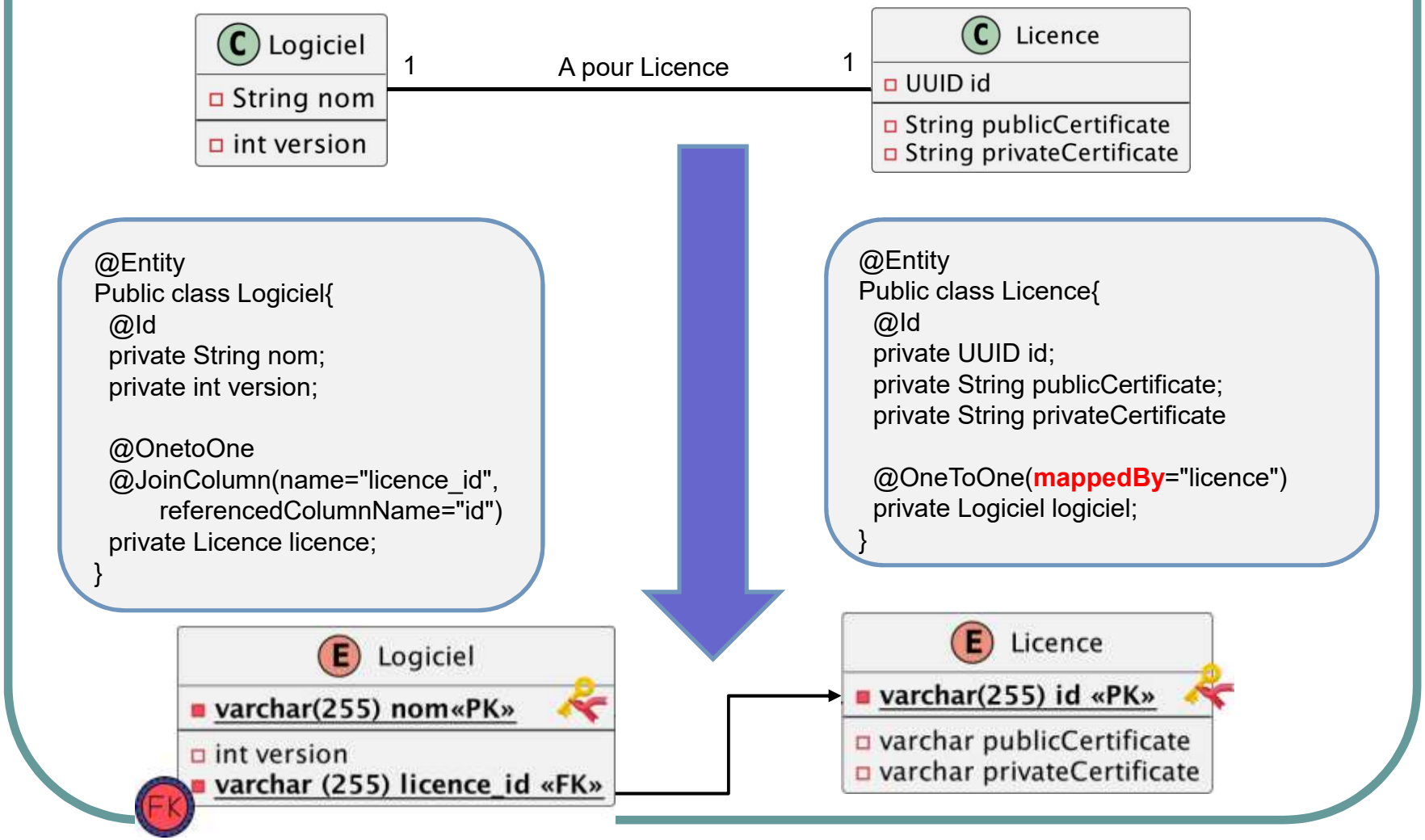
```
@Entity
Public class Logiciel{
  …
  @OnetoOne
private Licence licence;
}
```

```
@Entity
Public class Licence{
  …
  @OneToOne
  private Logiciel logiciel;
}
```

- Les deux entités se référencent mutuellement:
  - Chaque table possède une clé étrangère vers l'autre table
  - Pas de bijection mais deux relations indépendantes

- **JPA définit une relation inverse de l'autre.**

@OneToOne, bidirectionnelle

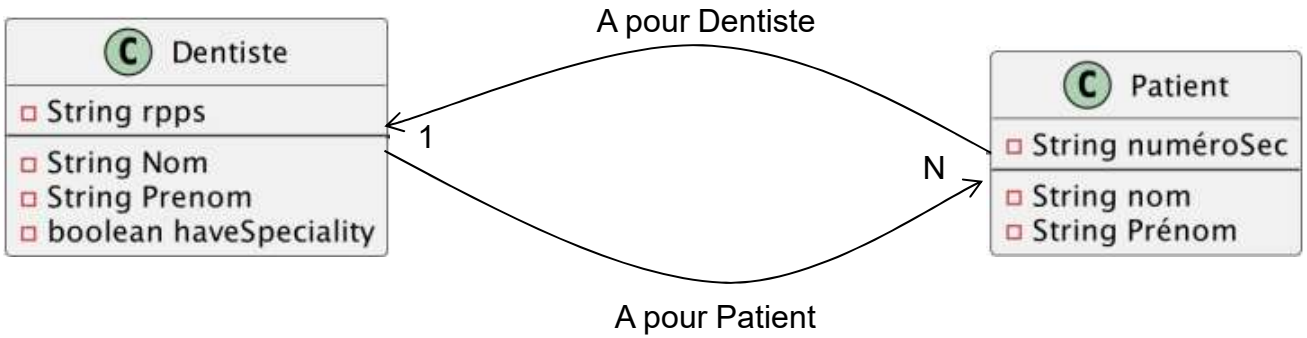


Relation OneToOne 1-1

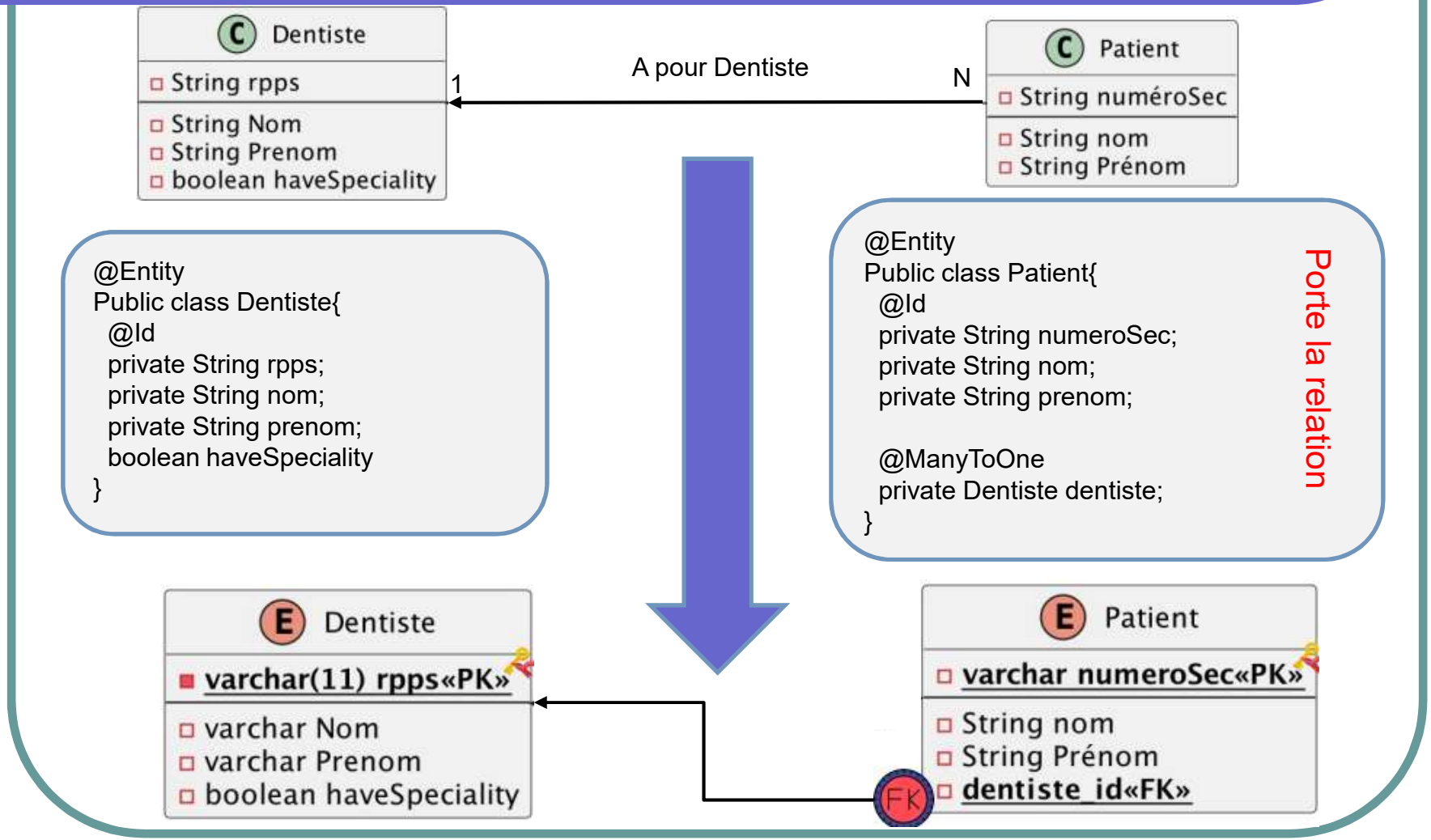
- L'entité dont la table possède la FK définit la relation principale
  - Elle référence l'autre entité
  - Annotation @JoinColumn pour spécifier la jointure avec la table de l'entité référencée
- L'entité référencée explicite la relation inverse
  - Pas de référence directe vers l'autre entité
  - Paramètre mappedBy sur l'attribut qui porte la relation dans l'annotation,

Relation ManyToOne / OneToMany

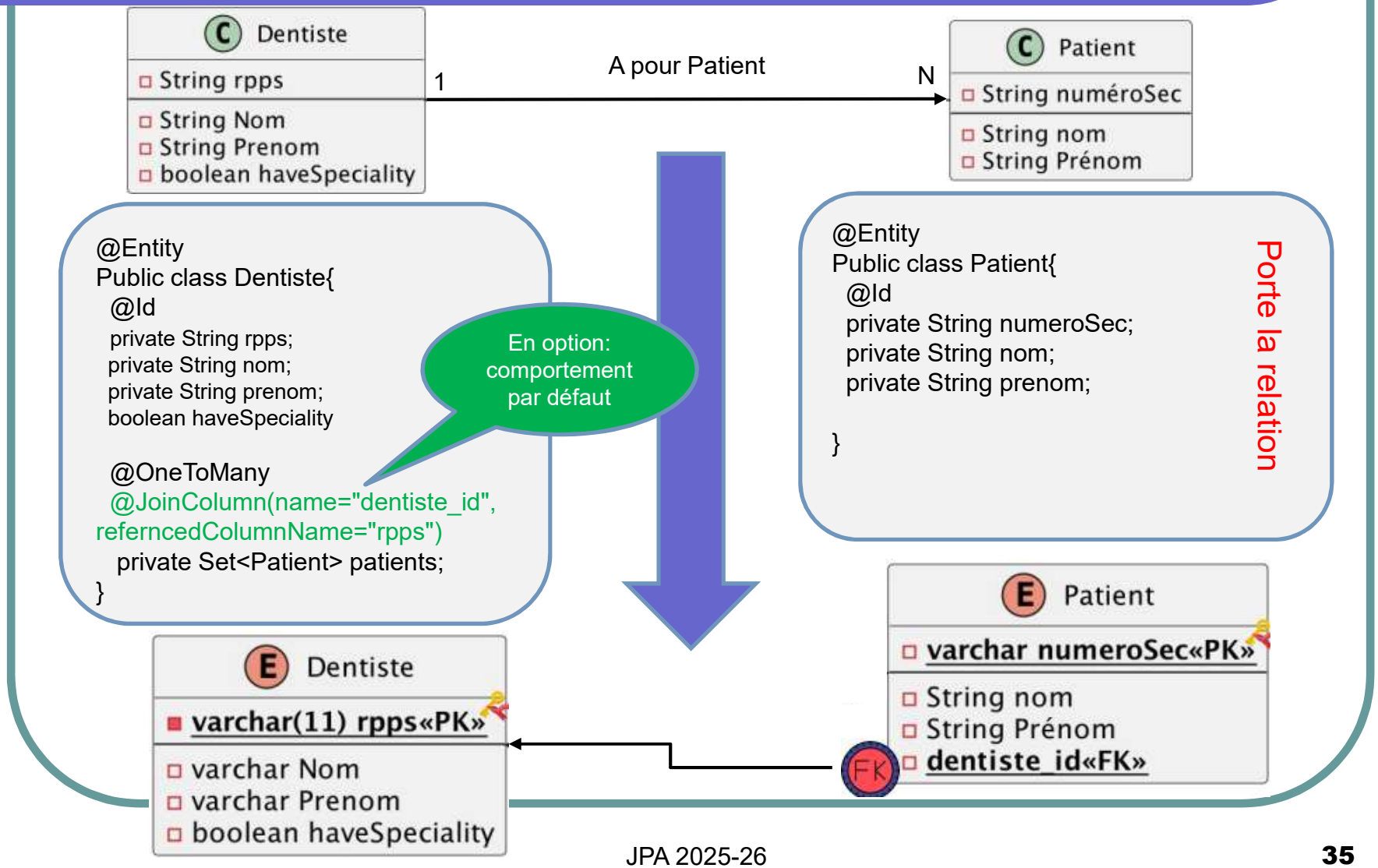
- Similaire à la relation OneToOne:
  - Unidirectionnelle ou bidirectionnelle
  - Donc l'entité qui implémente la clé étrangère (FK) dans sa table mappée possède la relation.



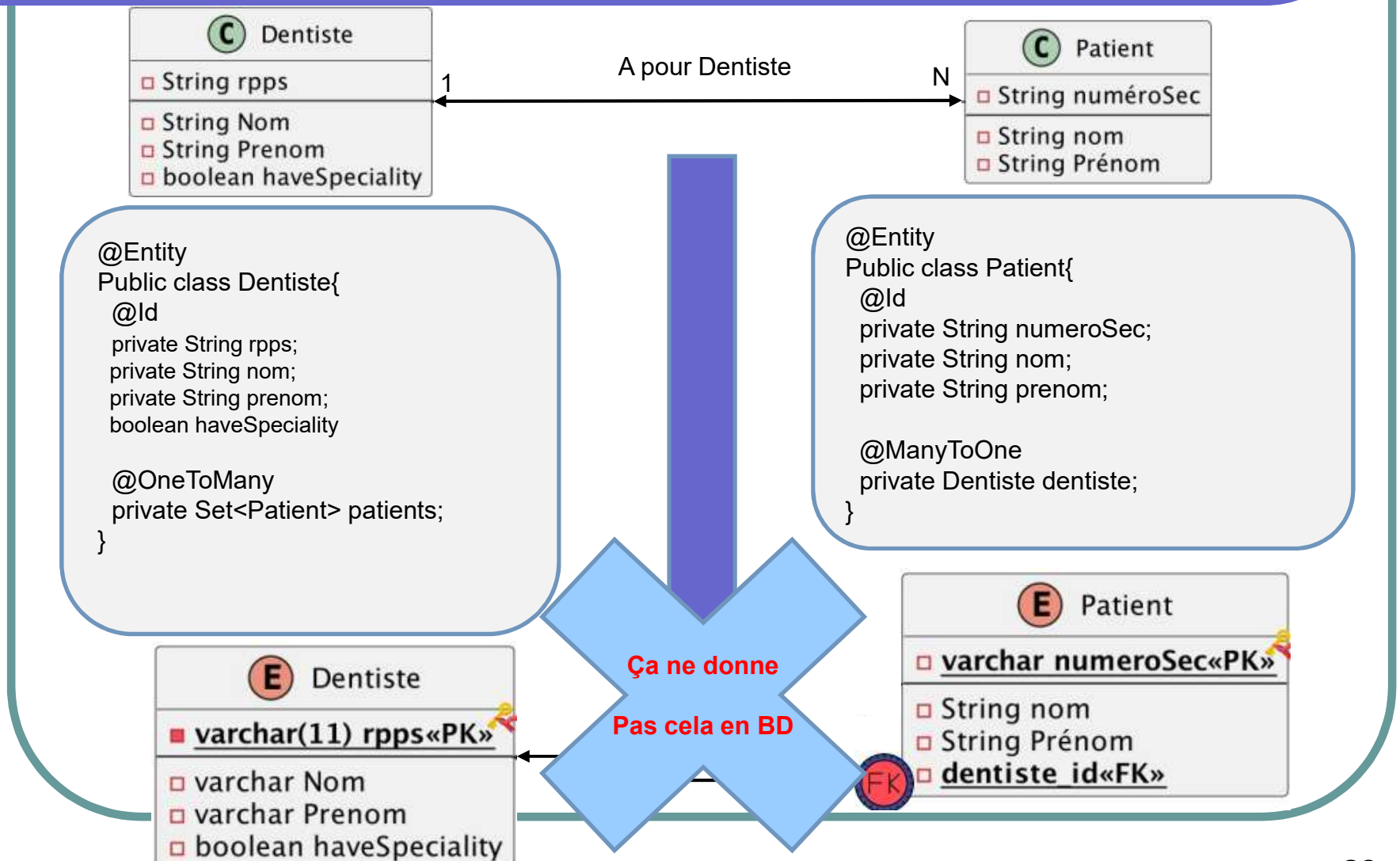
@ManyToOne, unidirectionnelle



@OneToMany, unidirectionnelle

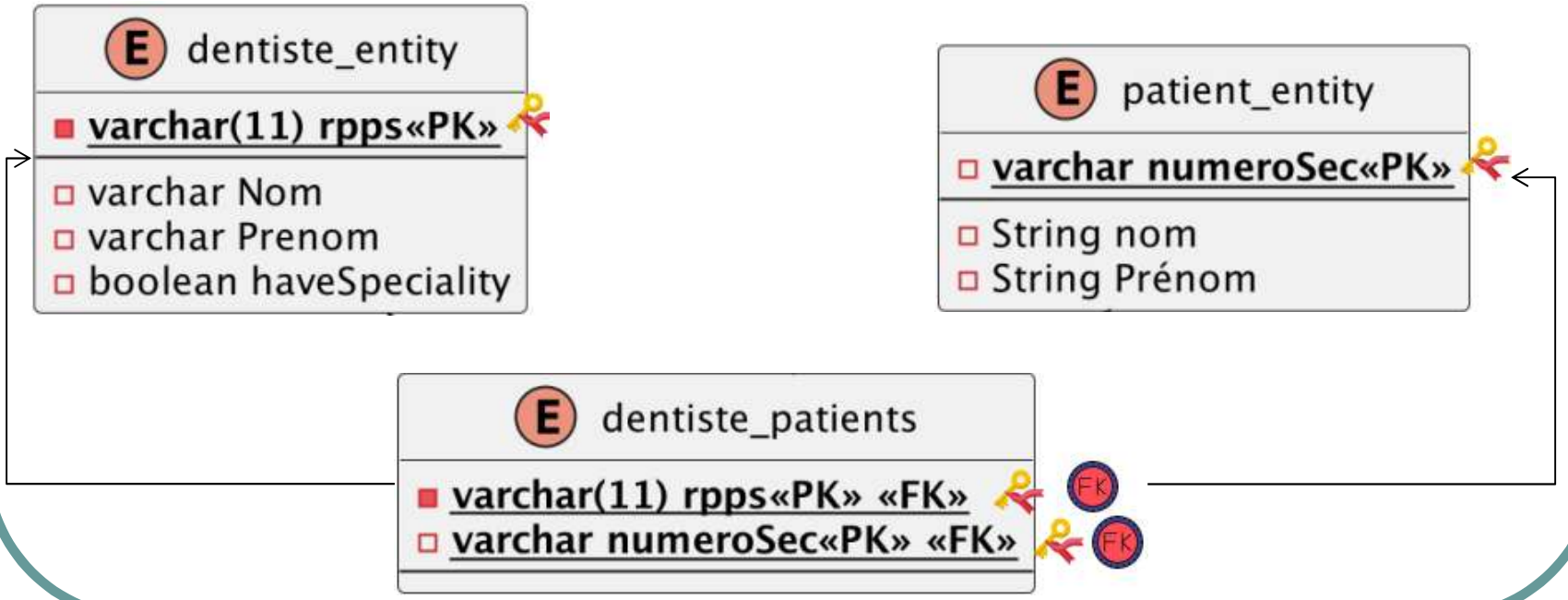


@ManyToOne + @OneToMany

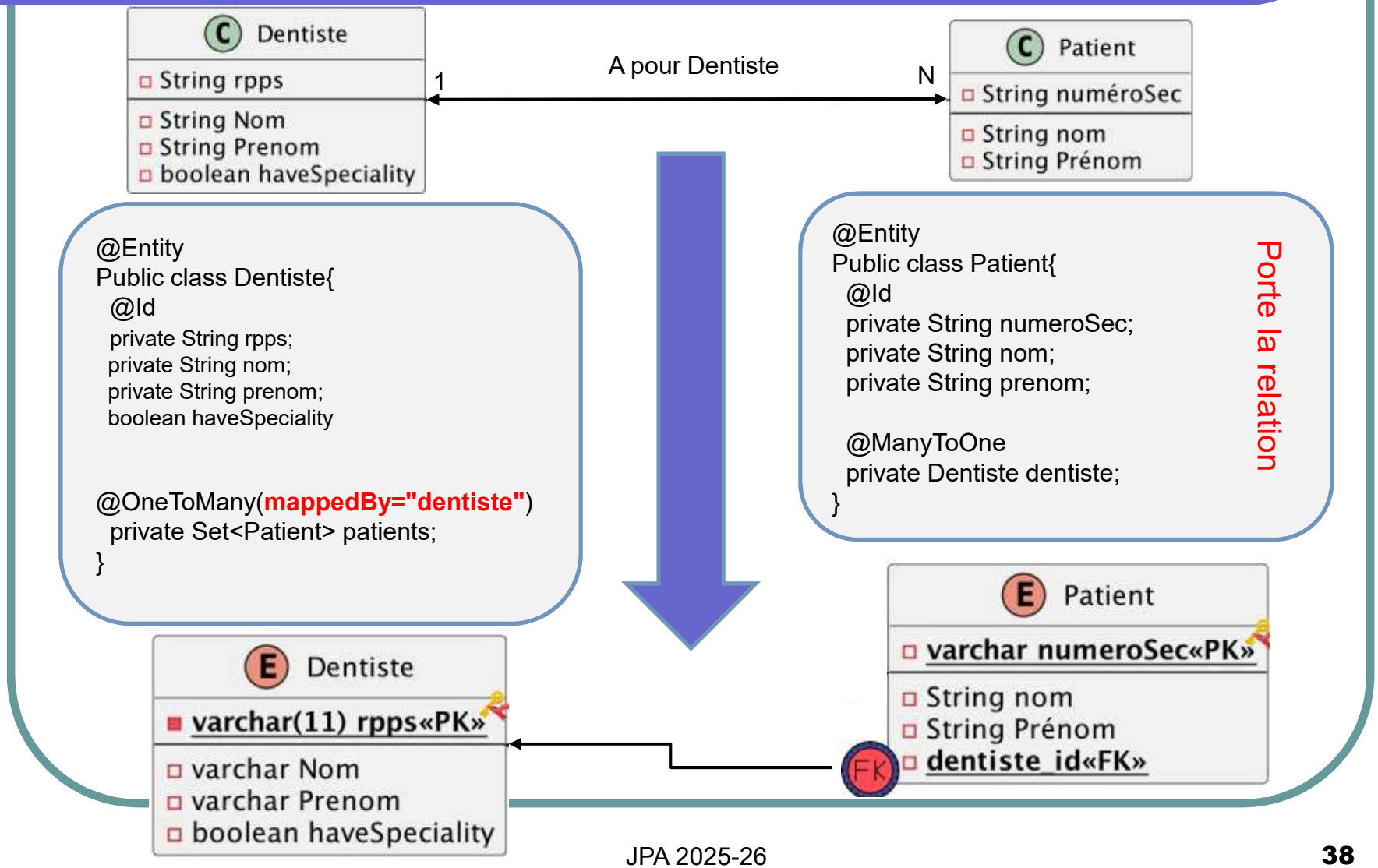


@ManyToOne + @OneToMany

- Représentation réelle en BD:
  - Pas de relation simple FK / PK
  - Mais une table d'association (possible mais conceptuellement "borderline")



@ManyToOne + @OneToMany bidirectionnelle



Relation 1:N, autre exemple

```
@Entity
public class Personne  {
   @Id
   private Integer id;
   …
   @OneToOne(cascade = CascadeType.ALL, mappedBy = "personne")
   private PersonneDetail personneDetail;

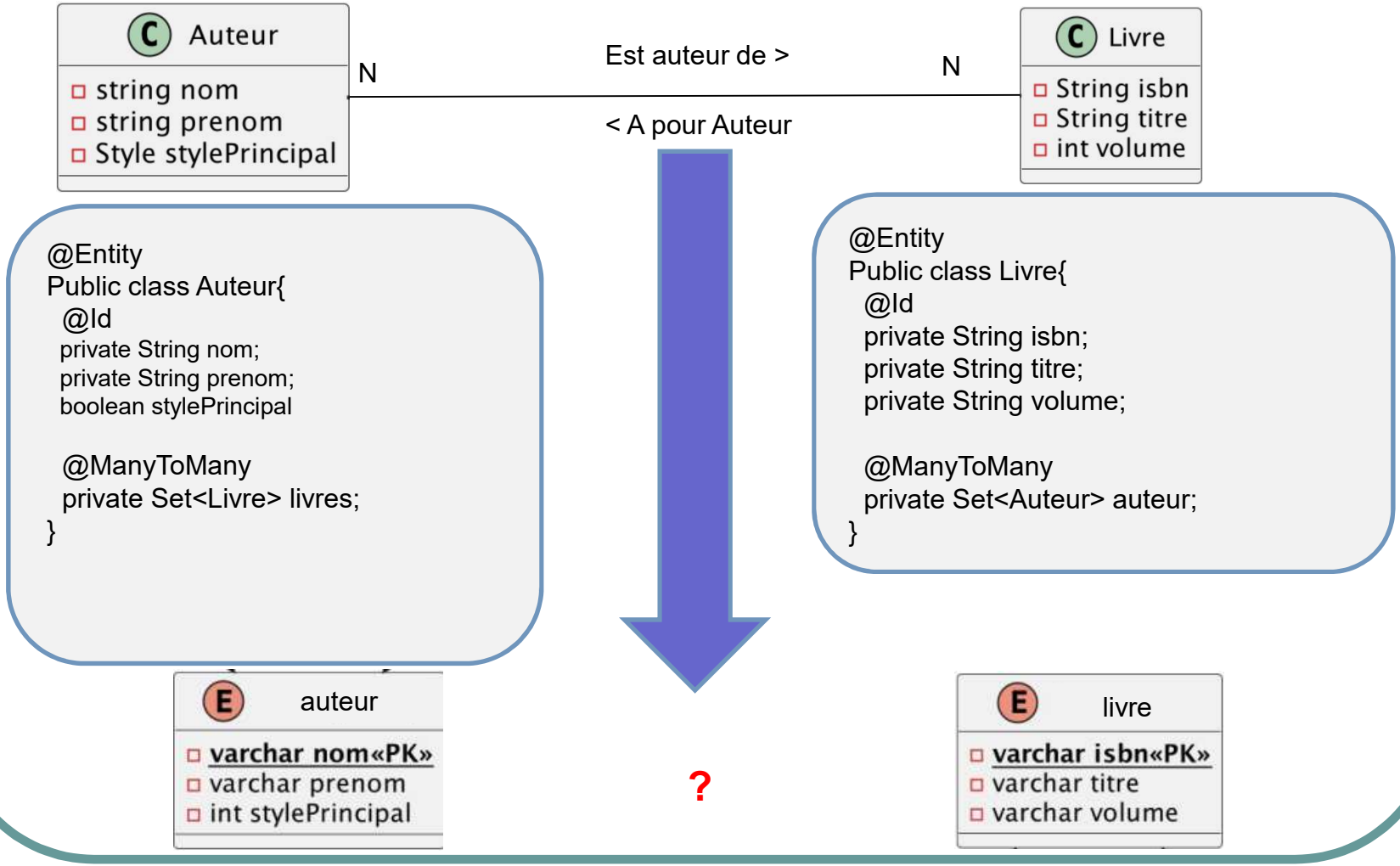
   @OneToMany(mappedBy = "personne")
   private List<Telephone> telephoneList;
   …
}
```

Guide la propagation d'une insertion/modification/suppression sur les objets liés.



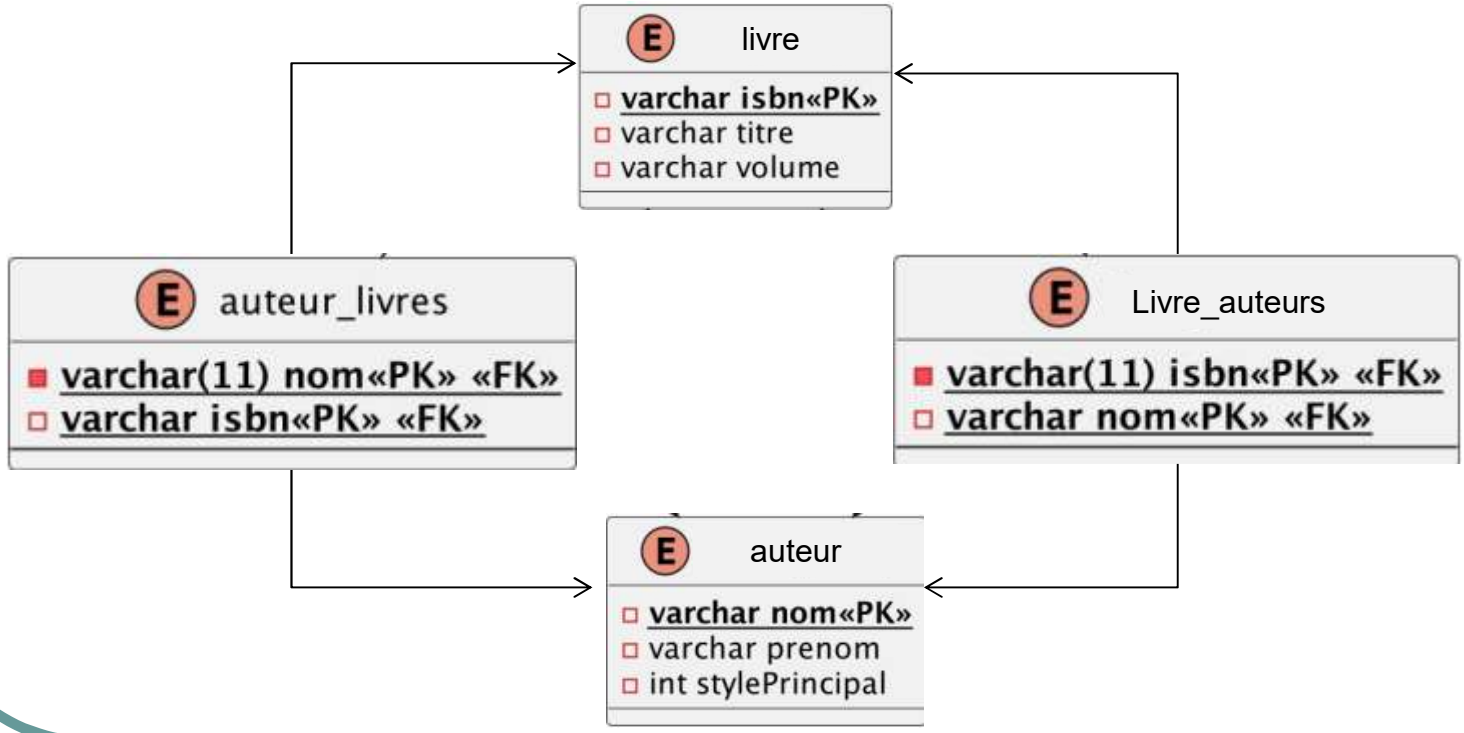
```
@Entity
public class Telephone {
   …
   @ManyToOne
   private Personne personne;
   … }
```

@ManyToMany

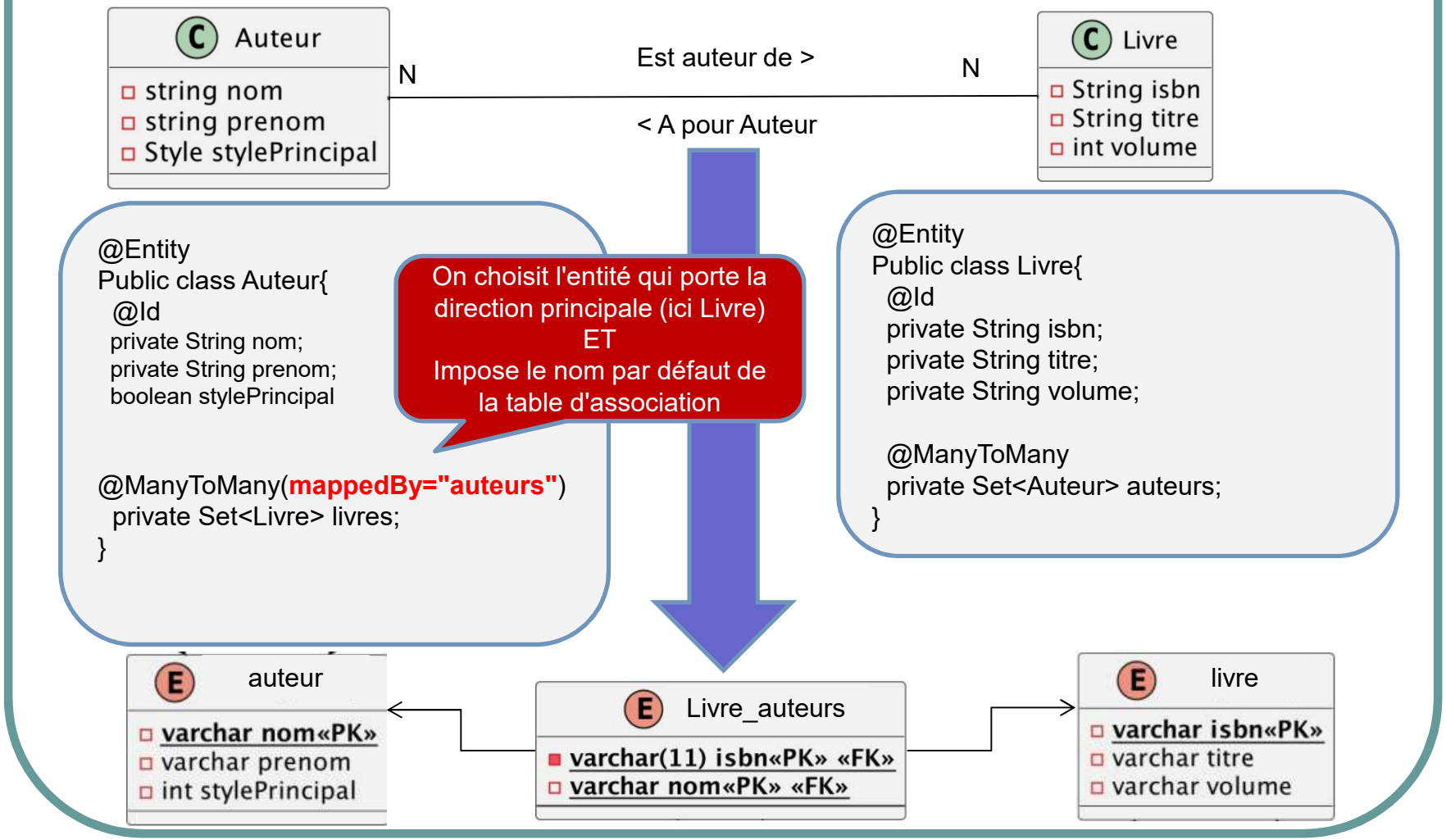

Auteur
string nom
string prenom
Style stylePrincipal
N
Est auteur de >
< A pour Auteur
N
Livre
String isbn
String titre
int volume
@Entity
Public class Auteur{
@Id
private String nom;
private String prenom;
boolean stylePrincipal
@ManyToMany
private Set<Livre> livres;
}
@Entity
Public class Livre{
@Id
private String isbn;
private String titre;
private String volume;
@ManyToMany
private Set<Auteur> auteur;
}
auteur
varchar nom«PK»
varchar prenom
int stylePrincipal
?
livre
varchar isbn«PK»
varchar titre
varchar volume

@ManyToOne + @OneToMany

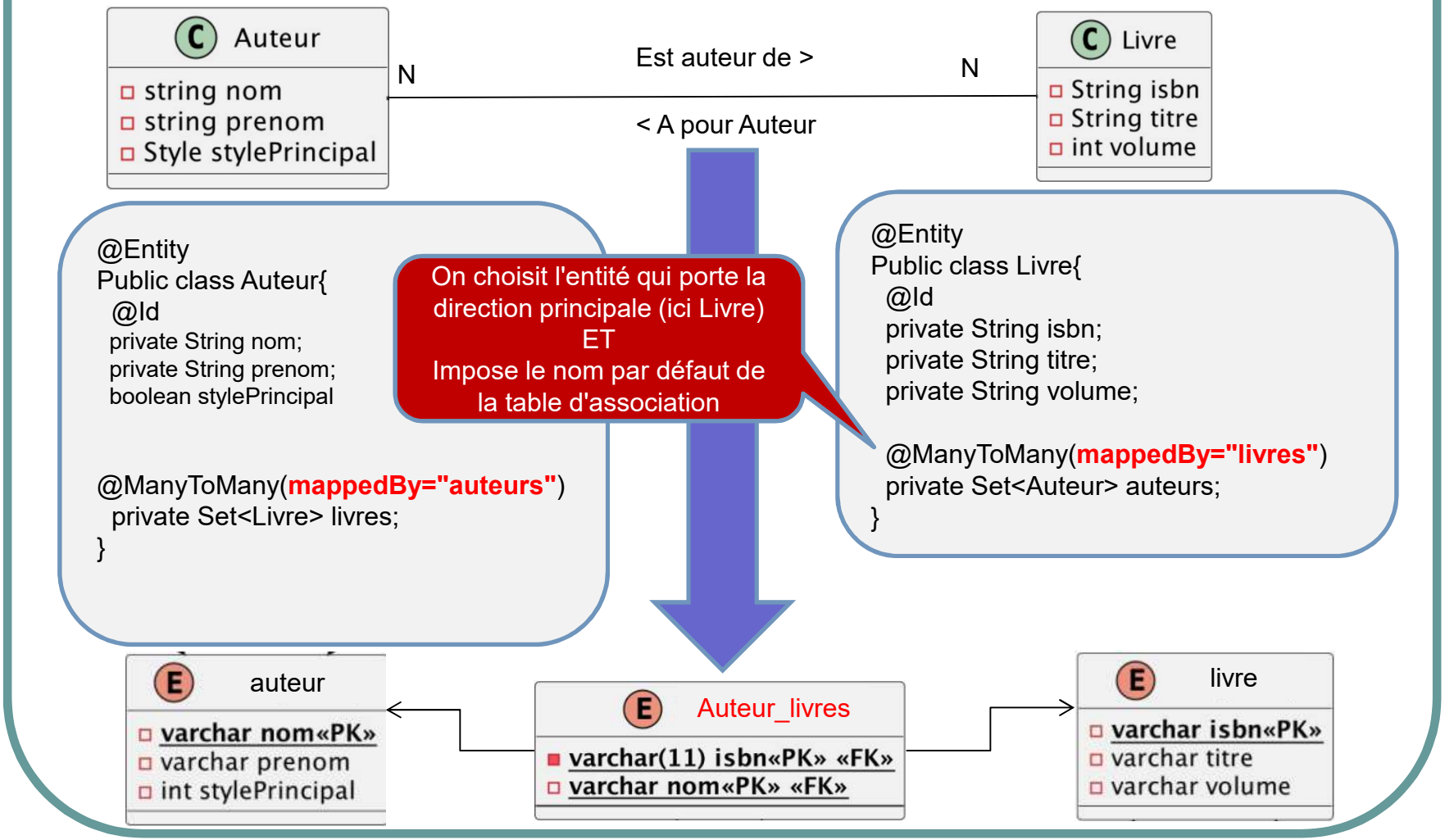
- Représentation réelle en BD:
  - Chaque mapping définit sa table d'association **unidirectionnelle**
  - Ok si une relation directionnelle, mais pas les deux !



@ManyToMany


Auteur
string nom
string prenom
Style stylePrincipal
N
Est auteur de >
< A pour Auteur
N
Livre
String isbn
String titre
int volume
@Entity
Public class Auteur{
@Id
private String nom;
private String prenom;
boolean stylePrincipal
@ManyToMany(mappedBy="auteurs")
private Set<Livre> livres;
}
On choisit l'entité qui porte la direction principale (ici Livre)
ET
Impose le nom par défaut de la table d'association
@Entity
Public class Livre{
@Id
private String isbn;
private String titre;
private String volume;
@ManyToMany
private Set<Auteur> auteurs;
}
auteur
varchar nom«PK»
varchar prenom
int stylePrincipal
Livre_auteurs
varchar(11) isbn«PK» «FK»
varchar nom«PK» «FK»
livre
varchar isbn«PK»
varchar titre
varchar volume

@ManyToMany


Auteur
string nom
string prenom
Style stylePrincipal
N
Est auteur de >
< A pour Auteur
N
Livre
String isbn
String titre
int volume
@Entity
Public class Auteur{
@Id
private String nom;
private String prenom;
boolean stylePrincipal
@ManyToMany(mappedBy="auteurs")
private Set<Livre> livres;
}
On choisit l'entité qui porte la direction principale (ici Livre)
ET
Impose le nom par défaut de la table d'association
@Entity
Public class Livre{
@Id
private String isbn;
private String titre;
private String volume;
@ManyToMany(mappedBy="livres")
private Set<Auteur> auteurs;
}
auteur
varchar nom«PK»
varchar prenom
int stylePrincipal
Auteur_livres
varchar(11) isbn«PK» «FK»
varchar nom«PK» «FK»
livre
varchar isbn«PK»
varchar titre
varchar volume

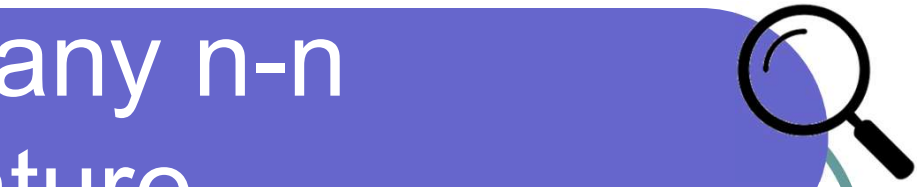
Relation ManyToMany n-n

- Approche similaire avec entité propriétaire et secondaire
  - Une table de jointure est générée avec une convention de nommage précise (sans nommage explicite): *table EntitéPropriétaire_EntitéSecondaire*

```
@Entity
public class Pays {
  @Id
  private Long pays_id;
  …
  @ManyToMany
  private List<Langue> lstLangue;
  …
}
```

```
@Entity
public class Langue {
  @Id
  private Long langue_id;
  …
  @ManyToMany(mappedBY="lstLangue")
  private List<Pays> lstPays;
  …
}
```

Relation ManyToMany n-n Données sur la jointure



- Si la relation porte des attributs le mapping @ManyToMany est impossible directement
  - Remplacer par deux relations @ManyToOne dans une classe de jointure explicite
  - Gérer une clé composite @EmbeddedId
- Exemple: Relation n-n entre Personne et Adresse avec le status d'adresse principale, dates de début et fin…



Relation ManyToMany n-n
Données sur la jointure

Classe d'association

Gestion de clé primaire composite

```
@Embeddable
class PersonneAdressePK
                implements Serializable{
   @Column(name = "id_adresse")
   private int idAdresse;
   @Column(name = "id_personne")
   private int idPersonne; }
```

```
@Entity
public class PersonneAdresse {

   @EmbeddedId
   protected PersonneAdressePK personneAdressePK;
   private boolean principale;
   @Temporal(TemporalType.DATE)
   private Date debut;
   @Temporal(TemporalType.DATE)
   private Date fin;
   @JoinColumn(name = "id_adresse", referencedColumnName = "id", insertable = false, updatable = false)
   @ManyToOne(optional = false)
   private Adresse adresse;

   @JoinColumn(name = "id_personne", referencedColumnName = "id", insertable = false, updatable = false)
   @ManyToOne(optional = false)
   private Personne personne;
   …
}
```

Les attributs de l'associations

Relation ManyToMany n-n
Données sur la jointure

```
Entity
public class Personne  {
    @Id
    private Integer id;
    …
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "personne")
    private List<PersonneAdresse> personneAdresseList;
    …
}
```

```
@Entity
public class Adresse {
    @Id
    private Integer id;
    …
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "adresse")
    private List<PersonneAdresse> personneAdresseList;
    …
}
```

Les relations dans le détail

Deux liens intéressants pour comprendre dans le détail toutes les subtilités du mapping des relations entre classes:

- https://blog.paumard.org/cours/jpa/chap03-entite-relation.html
- https://www.baeldung.com/learn-jpa-hibernate

- https://koor.fr/Java/TutorialJEE/jee_jpa_one_to_many.wp

Relation bidirectionnelle
Attention aux relations cycliques !

- Les bidirections doivent être utilisées
  - Si les deux directions sont nécessaires dans l'application
  - Si le modèle l'impose
- Les bidirections sont couteuses
  - En MàJ par JPA
  - En jointure
- **Les bidirections entrainent un bouclage dans le "fetching" des données**
  - Les téléphones des personnes avec leurs téléphones des personnes avec leurs téléphones…
  - Utiliser l'API jackon pour le post-traitement JSON (bidouillage)
  - **Utiliser des objets de transferts (DTO)**
  - **Utiliser un accès LAZY (et non pas EAGER)**

Configurer le chargement des données

- Définit la stratégie d'accès aux données
  - LAZY (par défaut): charge les données au moment de l'utilisation.
    - + Les entités en relation avec l'entité chargé ne sont pas chargés en mémoire
    - + Plus efficace, plus économe en mémoire
    - - Navigation ralentie,
    - - Maintenir la session (transaction) sur l'accès complet
  - EAGER: charge les données liées en réalisant toutes les jointures requises
    - + Navigation rapide
    - + Possibilité de travailler en mode déconnecté (hors session)
    - + Première accès lent
    - + Consommation / saturation mémoire

Configurer le chargement des données

- Configuration EAGER par défaut sur les attributs simples d'une entité
- Configuration LAZY par défaut sur les attributs définissant une relation
- Exemple de mise en place

```
@Basic(fetch = FetchType.EAGER)
Private String nom;

@OneToOne(fetch = FetchType.EAGER)
private PersonneDetail personneDetail;

@OneToMAny(fetch = FetchType.LAZY)
private List<Telephone> telephoneList;
```

Attention à l'égalité d'objets

- Chaque entité doit redéfinir les méthodes equals() et hashcode()
  - Pour comparer des entitées
  - Pour ajouter des entités dans des listes
- Implémentation par défaut limitée
  - Deux objets identiques issus de deux sessions sont différents !
  - Deux objets identiques créés dans et hors sessions sont différents !
- Implémentation par redéfinition
  - Soit basée sur les clés primaires: les Ids ne doivent pas être générés par la BD, ils doivent être sauvés après la création.
  - Soit basée sur des champs métiers: très utilisés sur des champs métiers possèdant la propriété UNIQUE (NumSecu, email, etc.)

Attention à l'égalité d'objets

- Exemple de redéfinition pour une classe Personne

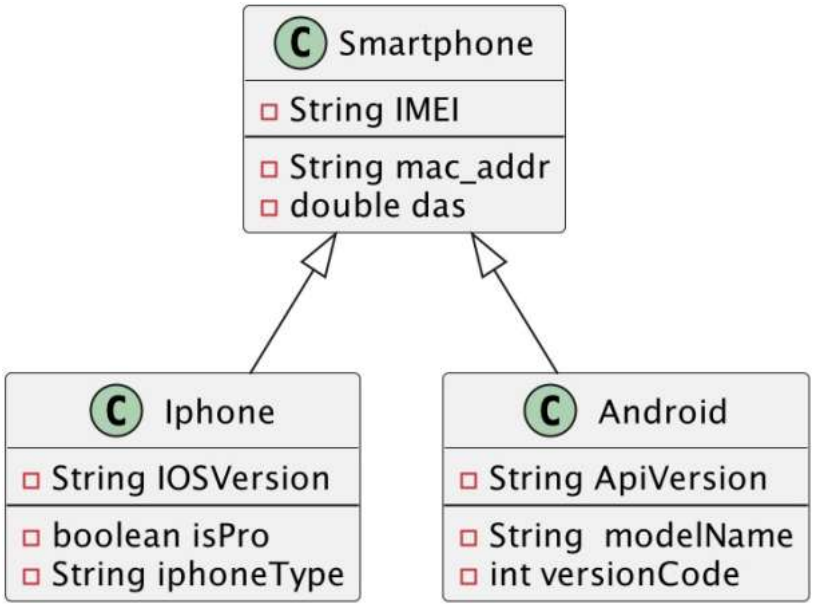
```
@Override
  public int hashCode() {
      int hash = 0;
      hash += (id != null ? id.hashCode() : 0);
      return hash;
  }

  @Override
  public boolean equals(Object object) {
      // TODO: Warning - this method won't work in the case the id fields are not set
      if (!(object instanceof Personne)) {
          return false;
      }
      Personne other = (Personne) object;
      if ((this.id == null && other.id != null) || (this.id != null && !this.id.equals(other.id))) {
          return false;
      }
      return true;
  }
```

Héritage (@Inheritance)

Notion d'héritage

- Le concept n'existe pas en relationnel
- Plusieurs stratégies d'implémentation:
  - 1 seule table
  - 1 table par classes
- JPA permet de choisir sa stratégie:
  - Performance
  - Cas d'utilisation
  - Simplification du code



3 stratégies d'implémen pour l'héritage



Table unique

Utiliser un champ de la table pour différencier la classe fille

+ performance: pas de jointure
+ schéma simple
+ requêtes simples
- Loin de la modèlisation OO
- Beaucoup de colonnes (nullable)
- Schéma statique

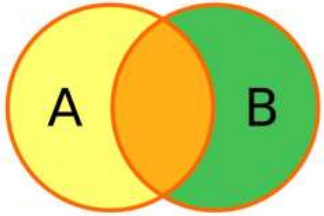


Table de jointure

Créer une table jointure pour différencier la classe fille

+ modèle normalisé
+ pas de colonne inutile
+ schéma évolutif
- Performance: jointures
- Requêtes complexes



Une table par classe

Créer une table par classe fille

+ performance
+ pas de colonnes inutiles
+ requêtes simples
- Difficile d'accéder aux parents
- Performance: si UNION
- Gestion Id (partagé)

Héritage: stratégie SINGLE_TABLE

Classe mère

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type")
public abstract class Smartphone {
   @Id
   private String imei;
   private String mac_addr;
   private double das;
}
```

→ `@DiscriminatorColumn(name="type")`

↓

Créer la colonne qui va permettre la différentiation

Classe fille

```
@Entity
@DiscriminatorValue("I")
public class Iphone extends Smartphone{
   String IosVersion;
   boolean isPro;
   String iphoneType;
}
```

`@DiscriminatorValue("")`

↓

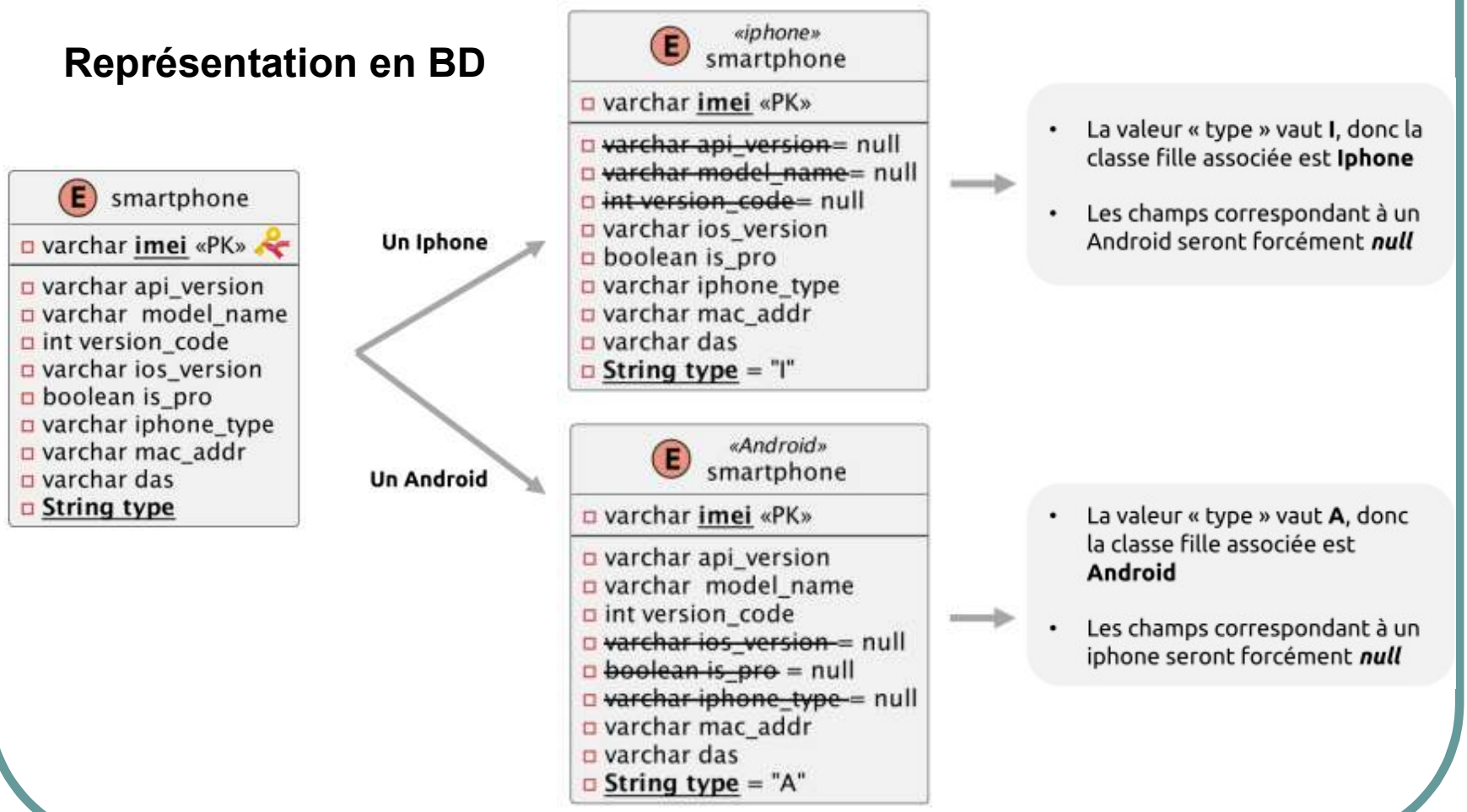
Permet l'association entre la colonne type et ses valeurs

Classe fille

```
@Entity
@DiscriminatorValue("A")
public class Android extends Smartphone{
   String ApiVersion;
   String modelName;
   int versionCode;
}
```

Héritage: stratégie SINGLE_TABLE

Représentation en BD



- La valeur « type » vaut **I**, donc la classe fille associée est **Iphone**
- Les champs correspondant à un Android seront forcément ***null***

- La valeur « type » vaut **A**, donc la classe fille associée est **Android**
- Les champs correspondant à un iphone seront forcément ***null***

Héritage: stratégie JOINED_TABLE

Classe mère

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
@DiscriminatorColumn(name="type")
public abstract class Smartphone {
   @Id
   private String imei;
   private String mac_addr;
   private double das;
}
```

Classe fille

```
@Entity
@DiscriminatorValue("I")
public class Iphone extends Smartphone{
   String IosVersion;
   boolean isPro;
   String iphoneType;
}
```

```
@DiscriminatorValue("")
```



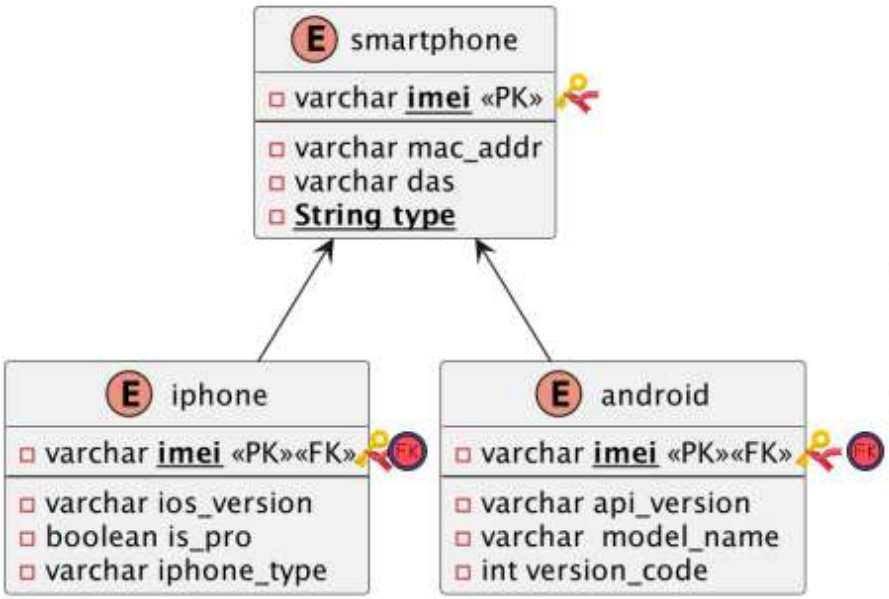
Permet l'association entre la colonne type et ses valeurs

Classe fille

```
@Entity
@DiscriminatorValue("A")
public class Android extends Smartphone{
   String ApiVersion;
   String modelName;
   int versionCode;
}
```

Héritage: stratégie JOINED_TABLE

Représentation en BD



- *Si **type** vaut **I**, alors une requête avec une jointure sur la table **iphone** va être faite pour récupérer toutes les informations*

- *Si **type** vaut **A**, alors une requête avec une jointure sur la table **Android** va être faite pour récupérer toutes les informations*

Héritage: stratégie TABLE_PER_CLASS

Classe mère

```
@MappedSuperclass
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Smartphone {
   @Id
   private String imei;
   private String mac_addr;
   private double das;
}
```

Classe fille

```
@Entity
public class Iphone extends Smartphone{
   String iosVersion;
   boolean isPro;
   String iphoneType;
}
```

@MappedSuperclass

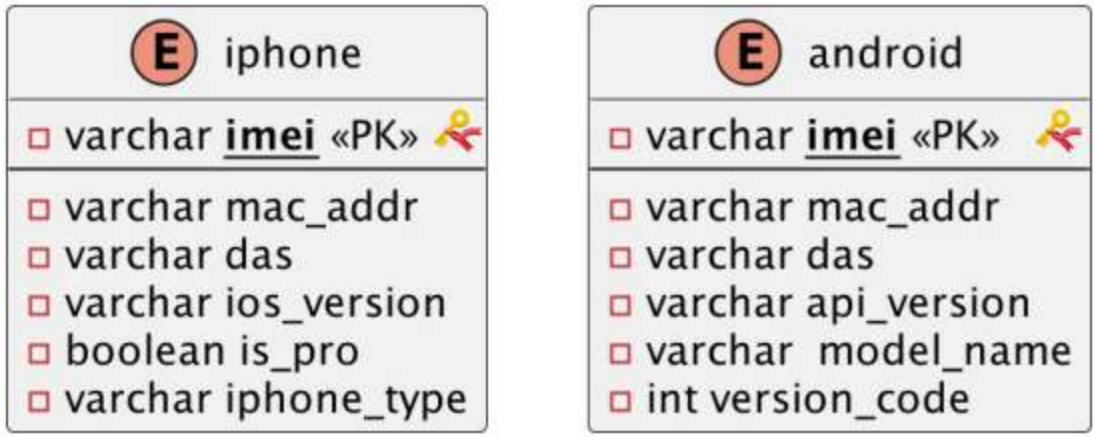
Permet de donner l'indication que la classe mère ne peut jamais être concrète

Classe fille

```
@Entity
public class Android extends Smartphone{
   String ApiVersion;
   String modelName;
   int versionCode;
}
```

Héritage: stratégie TABLE_PER_CLAS

Représentation en BD



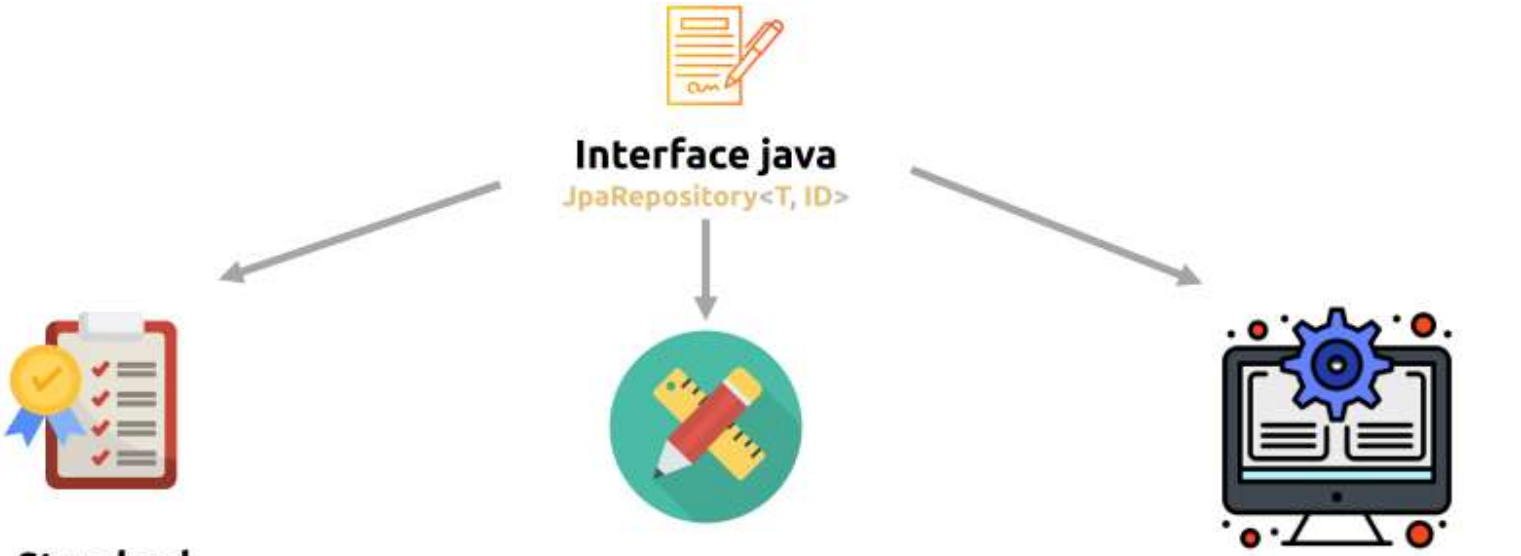


Ici une table par classe est créée, et aucune pour la classe mère.

JPA Repositories

DAO façon Spring Data

JPA Repository: le DAO tout-en-un



Standard

- save()
- findAll()
- findAllById()
- getOne()
- delete

Hérite de CrudRepository & PagingAndSortingRepository

Création de requête par interpolation du nom de la fonction

Création de requête JPQL, voir même SQL

JPA Repository: Implémentations

```
@Repository
public interface DentisteRepository extends JpaRepository<DentisteEntity,, String>{

   Optional<DentisteEntity> findDentisteByNomAndPrenom(String nom, String prenom);

   @Query("select d from Dentiste d where d.nom = ?1 and d.prenom = ?2")
   DentisteEntity getWithJpqlQuery(String nom, String prenom);

   @Query(value = "select * from public.dentiste_entity d where d.nom = ?1 and d.prenom = ?2",nativeQuery = true)
   DentisteEntity getWithSqlQuery(String nom, String prenom);
}
```

+ toutes les méthodes CRUD et d'accès paginé

- Attention les méthodes d'accès sont de bas niveau: pas de code métier complexe !

Requête JPQL

- 3 catégories de requêtes: SELECT, UPDATE, DELETE
- Construction similaire à une requête SQL
- La portée de la requête est une Classe et pas une requête:

Select p from Personne p

⇒ List<Personne>

Select p.nom from Personne p

⇒ List<String>

Select p.nom, p.prenom from Personne p

⇒ List<Object[]>

Requête JPQL

- Jointures: INNER JOIN & LEFT OUTER JOIN

```
Select p.nom, d.numsecu
From Personne p JOIN p.personneDetail d
ou
Select p.nom, d.numsecu
From Personne p INNER JOIN p.personneDetail d
ou
Select p.nom, p.personneDetail.numsecu
From Personne p
```



Et avec une jointure externe (sans surprise) pour conserver toutes les personnes:

```
Select p.nom, d.numsecu
From Personne p LEFT JOIN p.personneDetail d
ou
Select p.nom, d.numsecu
From Personne p LEFT OUTER JOIN p.personneDetail d
```

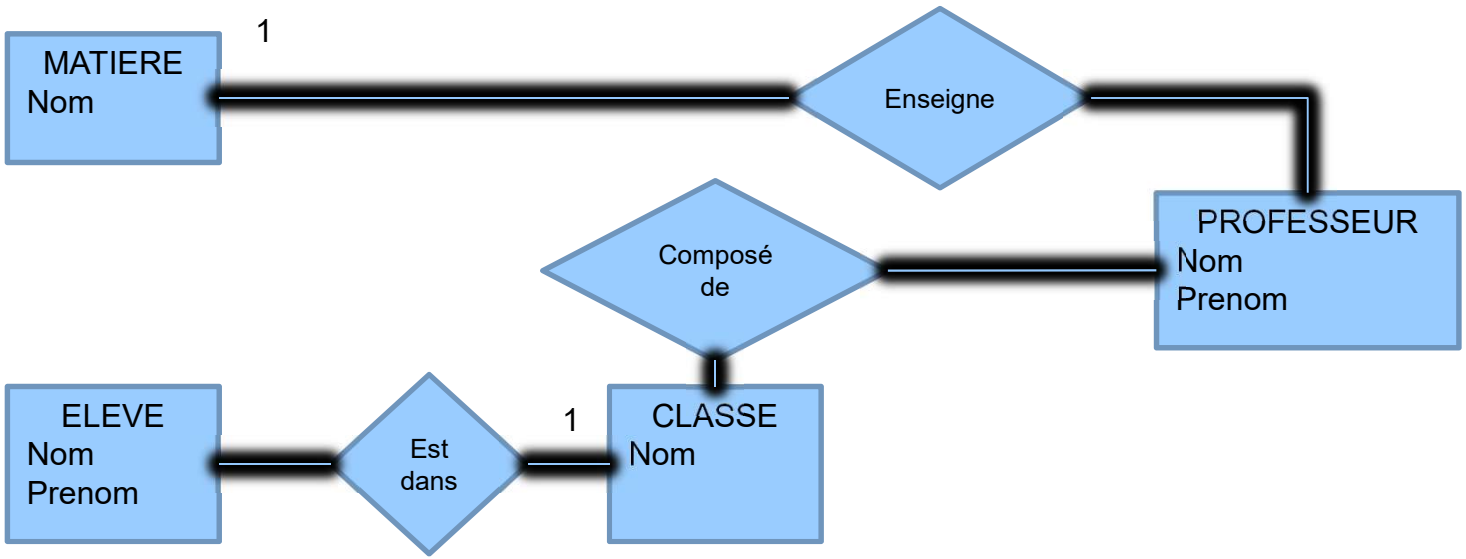
- Rien de spécial pour le reste (Where, Group by, having etc.)

Exemple Fil Rouge

… façon JPA

Contexte: Cas fil rouge

- Gestion simplifiée d'une école:
  - cette école est composée de classes ;
  - chaque classe est composée d'élèves ;
  - à chaque classe est attribué un ensemble de professeur
  - un professeur enseigne une et une seule matière mais peut exercer dans plusieurs classes.



Professeur & Matière

- Matiere

```
@Entity
public class Matiere {
  @Id @GeneratedValue
  private Long id;
  private String nom; }
```

```
CREATE TABLE MATIERE (
  ID   NUMBER(19) PRIMARY KEY,
  NOM  VARCHAR2(255)
);
```

- Professeur

```
@Entity
public class Professeur {
  @Id @GeneratedValue
  private Long id;
  private String nom;
  private String prenom;
  @ManyToOne
  private Matiere matiere; }
```

```
CREATE TABLE PROFESSEUR (
  ID          NUMBER(19) PRIMARY KEY,
  NOM         VARCHAR2(255),
  PRENOM      VARCHAR2(255),
  MATIERE_ID  NUMBER(19),
  FOREIGN KEY (MATIERE_ID) REFERENCES MATIERE(ID)
);
```

Classe & Elève

- Classe

```
@Entity
public class Classe {
  @Id
  private Long id;
  private String nom;
  @ManyToMany
  private Set<Professeur> professeurs; }
```

- Elève

```
@Entity
public class Eleve {
  @Id
  private Long id;
  private String nom;
  private String prenom;
  @ManyToOne
  private Classe classe; }
```

```
CREATE TABLE CLASSE (
  ID   NUMBER(19) PRIMARY KEY,
  NOM  VARCHAR2(255)
);
```

```
CREATE TABLE CLASSE_PROFESSEUR (
  CLASSE_ID      NUMBER(19) NOT NULL,
  PROFESSEUR_ID  NUMBER(19) NOT NULL,
  PRIMARY KEY (CLASSE_ID, PROFESSEUR_ID),
  FOREIGN KEY (CLASSE_ID) REFERENCES CLASSE(ID),
  FOREIGN KEY (PROFESSEUR_ID) REFERENCES PROFESSEUR(ID)
);
```

```
CREATE TABLE ELEVE (
  ID         NUMBER(19) PRIMARY KEY,
  NOM        VARCHAR2(255),
  PRENOM     VARCHAR2(255),
  CLASSE_ID  NUMBER(19),
  FOREIGN KEY (CLASSE_ID) REFERENCES CLASSE(ID)
);
```

Les Repositories pour Eleve & Matiere

```
public interface MatiereRepository extends
        JpaRepository<Matiere, Long> { }

public interface ClasseRepository extends
        JpaRepository<Classe, Long> { }
```

Repository Eleve

- **Méthode ajoutée sous forme de requête JPQL**

```
public interface EleveRepository extends JpaRepository<Eleve, Long> {
    // Compter le nombre d'élèves d'une classe donnée par son nom
    @Query("
            select count(e)
            from Eleve e
            where e.classe.nom = :nomClasse")
    long countByClasseNom(@Param("nomClasse") String nomClasse);
}
```

- **Méthode dérivée de son nom**

```
public interface EleveRepository extends JpaRepository<Eleve, Long> {
    long countByClasse_Nom(String nomClasse);
}
```

Repository Professeur

- **Méthode ajoutée sous forme de requête JPQL**

```
public interface ProfesseurRepository extends JpaRepository<Professeur, Long> {
   // Tous les professeurs d'une classe, cherchée par son nom
   @Query("""
         select p
         from Classe c
         join c.professeurs p
         where c.nom = :nomClasse
         """)
   List<Professeur> findAllByClasseNom(@Param("nomClasse") String nomClasse);}
```

- **Méthode dérivée de son nom**

```
public interface ProfesseurRepository extends JpaRepository<Professeur, Long> {
   List<Professeur> findByClasses_Nom(String nomClasse);}
```

To be continued…

Il faut tisser tout cela dans une architecture Micro-services de type Spring Boot!